

# SQL Window Functions

*A Complete Training Guide with Examples*

---

**Handout**

---

# 1. Introduction to Window Functions

---

Window functions perform calculations across a set of table rows that are related to the current row. Unlike aggregate functions, window functions do not collapse rows — they return a value for every row while still allowing you to compute aggregates across a defined 'window' of rows.

## Syntax Overview:

```
function_name(expression)
  OVER (
    [PARTITION BY column1, column2, ...]
    [ORDER BY column3 [ASC|DESC]]
    [ROWS|RANGE BETWEEN start AND end]
  )
```

## Key Clauses:

**OVER()** — Defines the window. Required for all window functions.

**PARTITION BY** — Divides rows into groups (like GROUP BY, but keeps all rows).

**ORDER BY** — Defines row ordering within each partition.

**ROWS/RANGE** — Specifies the window frame (subset of rows in the partition).

 **Note:** Window functions are processed AFTER WHERE, GROUP BY, and HAVING clauses. They cannot be used in WHERE or GROUP BY clauses.

# 2. Sample Dataset

---

All examples in this guide use the following employees table:

| emp_id | name  | department  | salary | hire_date  |
|--------|-------|-------------|--------|------------|
| 1      | Alice | Engineering | 90000  | 2020-01-15 |
| 2      | Bob   | Engineering | 85000  | 2019-03-10 |
| 3      | Carol | Engineering | 92000  | 2021-07-01 |
| 4      | Dave  | Marketing   | 70000  | 2018-06-20 |
| 5      | Eve   | Marketing   | 75000  | 2020-11-05 |
| 6      | Frank | Marketing   | 68000  | 2022-02-14 |
| 7      | Grace | HR          | 60000  | 2017-09-30 |
| 8      | Hank  | HR          | 62000  | 2019-12-01 |

Create this table using:

```
CREATE TABLE employees (  
    emp_id      INT,  
    name        VARCHAR(50),  
    department  VARCHAR(50),  
    salary      DECIMAL(10,2),  
    hire_date   DATE  
);  
  
INSERT INTO employees VALUES  
    (1, 'Alice', 'Engineering', 90000, '2020-01-15'),  
    (2, 'Bob', 'Engineering', 85000, '2019-03-10'),  
    (3, 'Carol', 'Engineering', 92000, '2021-07-01'),  
    (4, 'Dave', 'Marketing', 70000, '2018-06-20'),  
    (5, 'Eve', 'Marketing', 75000, '2020-11-05'),  
    (6, 'Frank', 'Marketing', 68000, '2022-02-14'),  
    (7, 'Grace', 'HR', 60000, '2017-09-30'),  
    (8, 'Hank', 'HR', 62000, '2019-12-01');
```

## 3. Ranking Functions

Ranking functions assign a rank or number to each row within a partition, based on ORDER BY.

### 3.1 ROW\_NUMBER()

Assigns a unique sequential integer to each row within a partition. No ties — every row gets a distinct number.

```
SELECT name, department, salary,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees;
```

Result:

| name  | department  | salary | row_num |
|-------|-------------|--------|---------|
| Carol | Engineering | 92000  | 1       |
| Alice | Engineering | 90000  | 2       |
| Bob   | Engineering | 85000  | 3       |
| Eve   | Marketing   | 75000  | 1       |
| Dave  | Marketing   | 70000  | 2       |
| Frank | Marketing   | 68000  | 3       |
| Hank  | HR          | 62000  | 1       |
| Grace | HR          | 60000  | 2       |

### 3.2 RANK()

Assigns a rank to each row. Rows with equal values receive the SAME rank, and gaps appear after ties.

```
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
FROM employees;
```

 **Note:** If two employees tie at rank 1, the next employee gets rank 3 (rank 2 is skipped). This is the key difference from DENSE\_RANK().

### 3.3 DENSE\_RANK()

Like RANK(), but without gaps. After a tie at rank 1, the next distinct value gets rank 2.

```
SELECT name, department, salary,
```

```

RANK()          OVER (ORDER BY salary DESC) AS rnk,
DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;

```

Comparison: RANK vs DENSE\_RANK (if Carol and Alice tied at 90000):

| name  | salary | RANK() | DENSE_RANK() |
|-------|--------|--------|--------------|
| Carol | 90000  | 1      | 1            |
| Alice | 90000  | 1      | 1            |
| Bob   | 85000  | 3      | 2            |
| Eve   | 75000  | 4      | 3            |

### 3.4 NTILE(n)

Divides rows into n approximately equal buckets and assigns a bucket number to each row. Useful for percentile analysis.

```

-- Divide employees into 4 salary quartiles
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;

```

Result:

| name  | salary | quartile |
|-------|--------|----------|
| Carol | 92000  | 1        |
| Alice | 90000  | 1        |
| Bob   | 85000  | 2        |
| Eve   | 75000  | 2        |
| Dave  | 70000  | 3        |
| Frank | 68000  | 3        |
| Hank  | 62000  | 4        |
| Grace | 60000  | 4        |

## 4. Aggregate Window Functions

Standard aggregate functions (SUM, AVG, COUNT, MIN, MAX) become window functions when used with OVER(). Unlike GROUP BY, all original rows are preserved.

### 4.1 Running Totals with SUM()

Calculate the running (cumulative) total of salaries ordered by hire date:

```
SELECT name, department, salary, hire_date,
       SUM(salary) OVER (
         PARTITION BY department
         ORDER BY hire_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM employees
ORDER BY department, hire_date;
```

Result (Engineering only):

| name  | department  | salary | hire_date  | running_total |
|-------|-------------|--------|------------|---------------|
| Bob   | Engineering | 85000  | 2019-03-10 | 85000         |
| Alice | Engineering | 90000  | 2020-01-15 | 175000        |
| Carol | Engineering | 92000  | 2021-07-01 | 267000        |

### 4.2 Department Totals with SUM()

Show each employee's salary alongside the total salary of their department:

```
SELECT name, department, salary,
       SUM(salary) OVER (PARTITION BY department) AS dept_total,
       ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 1) AS
pct_of_dept
FROM employees;
```

| name  | department  | salary | dept_total | pct_of_dept |
|-------|-------------|--------|------------|-------------|
| Alice | Engineering | 90000  | 267000     | 33.7        |
| Bob   | Engineering | 85000  | 267000     | 31.8        |
| Carol | Engineering | 92000  | 267000     | 34.5        |
| Dave  | Marketing   | 70000  | 213000     | 32.9        |
| Eve   | Marketing   | 75000  | 213000     | 35.2        |

| name  | department | salary | dept_total | pct_of_dept |
|-------|------------|--------|------------|-------------|
| Frank | Marketing  | 68000  | 213000     | 31.9        |

### 4.3 Moving Average with AVG()

Calculate a 3-row moving average of salary (current row + 2 preceding rows):

```
SELECT name, salary,
       AVG(salary) OVER (
         ORDER BY emp_id
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg_3
FROM employees;
```

 **Note:** ROWS BETWEEN 2 PRECEDING AND CURRENT ROW means: start from 2 rows before the current row, end at the current row.

### 4.4 COUNT() — Partition Size

```
SELECT name, department,
       COUNT(*) OVER (PARTITION BY department) AS dept_headcount
FROM employees;
```

## 5. Offset / Navigation Functions

These functions let you access values from different rows relative to the current row within the window.

### 5.1 LAG()

Returns the value from a previous row (default: 1 row before). Useful for period-over-period comparisons.

```
-- LAG(column, offset, default_value)
SELECT name, department, salary,
       LAG(salary, 1, 0) OVER (PARTITION BY department ORDER BY salary) AS
prev_salary,
       salary - LAG(salary, 1, 0) OVER (PARTITION BY department ORDER BY salary) AS
salary_diff
FROM employees;
```

| name  | department  | salary | prev_salary | salary_diff |
|-------|-------------|--------|-------------|-------------|
| Bob   | Engineering | 85000  | 0           | 85000       |
| Alice | Engineering | 90000  | 85000       | 5000        |
| Carol | Engineering | 92000  | 90000       | 2000        |
| Frank | Marketing   | 68000  | 0           | 68000       |
| Dave  | Marketing   | 70000  | 68000       | 2000        |
| Eve   | Marketing   | 75000  | 70000       | 5000        |

### 5.2 LEAD()

Returns the value from a following row (default: 1 row ahead). Useful to look ahead at what comes next.

```
-- Show each employee's salary and the next higher salary in their dept
SELECT name, department, salary,
       LEAD(salary) OVER (PARTITION BY department ORDER BY salary) AS next_salary
FROM employees;
```

| name  | department | salary | next_salary |
|-------|------------|--------|-------------|
| Frank | Marketing  | 68000  | 70000       |
| Dave  | Marketing  | 70000  | 75000       |
| Eve   | Marketing  | 75000  | NULL        |
| Grace | HR         | 60000  | 62000       |



| name | department | salary | next_salary |
|------|------------|--------|-------------|
| Hank | HR         | 62000  | NULL        |

### 5.3 FIRST\_VALUE() and LAST\_VALUE()

FIRST\_VALUE() returns the first value in the window frame. LAST\_VALUE() returns the last.

```
SELECT name, department, salary,
       FIRST_VALUE(name) OVER (
         PARTITION BY department ORDER BY salary DESC
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS highest_earner,
       LAST_VALUE(salary) OVER (
         PARTITION BY department ORDER BY salary DESC
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS lowest_salary
FROM employees;
```

 **Note:** Always use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING with LAST\_VALUE(). Without it, the default frame ends at the current row, which rarely gives the intended result.

### 5.4 NTH\_VALUE() ⚠ Not in SQL Server

NTH\_VALUE() returns the value from the Nth row in the window. It is available in MySQL 8+, PostgreSQL, and Oracle — but NOT in SQL Server. Use a DENSE\_RANK() workaround instead:

```
-- SQL Server workaround: Get the 2nd highest salary in each department
WITH ranked AS (
  SELECT department, salary,
         DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
  FROM employees
)
SELECT DISTINCT department,
       MAX(CASE WHEN rnk = 2 THEN salary END) AS second_highest_salary
FROM ranked
GROUP BY department;
```

 **Note:** SQL Server does not support NTH\_VALUE(). The DENSE\_RANK() + CASE pattern is the standard SQL Server equivalent.



## 6. Window Frame Specification

---

The frame clause defines which rows within the partition are included in the calculation for each current row.

| Frame Clause                                             | Meaning                                          |
|----------------------------------------------------------|--------------------------------------------------|
| ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW         | From first row to current row (cumulative)       |
| ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING         | From current row to last row                     |
| ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING | All rows in the partition                        |
| ROWS BETWEEN 2 PRECEDING AND CURRENT ROW                 | Current row + 2 rows before (3-row window)       |
| ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING                 | Previous, current, and next row (3-row centered) |
| RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW        | Like ROWS but includes ties with current         |

### ROWS vs RANGE

ROWS treats each row individually. RANGE treats rows with the same ORDER BY value as a group (peers).

```
-- If two rows have the same salary value:  
-- ROWS: each row counted individually  
-- RANGE: both rows included in each other's frame
```

```
SUM(salary) OVER (ORDER BY salary ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)  
SUM(salary) OVER (ORDER BY salary RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT  
ROW)
```

## 7. Real-World Practical Examples

---

### 7.1 Top N per Group (Top 2 earners per department)

```
WITH ranked AS (  
    SELECT name, department, salary,  
           DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk  
    FROM employees  
)  
SELECT name, department, salary, rnk  
FROM ranked  
WHERE rnk <= 2;
```

| name  | department  | salary | rnk |
|-------|-------------|--------|-----|
| Carol | Engineering | 92000  | 1   |
| Alice | Engineering | 90000  | 2   |
| Eve   | Marketing   | 75000  | 1   |
| Dave  | Marketing   | 70000  | 2   |
| Hank  | HR          | 62000  | 1   |
| Grace | HR          | 60000  | 2   |

### 7.2 Salary vs Department Average

```
SELECT name, department, salary,  
       ROUND(AVG(salary) OVER (PARTITION BY department), 0) AS dept_avg,  
       CASE  
           WHEN salary > AVG(salary) OVER (PARTITION BY department) THEN 'Above  
Average'  
           WHEN salary = AVG(salary) OVER (PARTITION BY department) THEN 'Average'  
           ELSE 'Below Average'  
       END AS salary_status  
FROM employees;
```

### 7.3 Detecting Consecutive Events (using LAG)

```
-- Find employees hired within 6 months of the previous hire  
SELECT name, hire_date,  
       LAG(hire_date) OVER (ORDER BY hire_date) AS prev_hire_date,
```

```
DATEDIFF(DAY,  
         LAG(hire_date) OVER (ORDER BY hire_date), hire_date) AS  
days_since_last_hire  
FROM employees  
ORDER BY hire_date;
```

## 7.4 Percentile with PERCENT\_RANK() and CUME\_DIST()

```
SELECT name, salary,  
       ROUND(PERCENT_RANK() OVER (ORDER BY salary) * 100, 1) AS percentile_rank,  
       ROUND(CUME_DIST()    OVER (ORDER BY salary) * 100, 1) AS cumulative_pct  
FROM employees;
```

PERCENT\_RANK():  $(\text{rank} - 1) / (\text{total rows} - 1)$  — ranges from 0 to 1.

CUME\_DIST():  $(\text{rows} \leq \text{current}) / \text{total rows}$  — proportion of rows at or below current.

## 8. Quick Reference Summary

---

| Function          | Category   | Description                                                                                    |
|-------------------|------------|------------------------------------------------------------------------------------------------|
| ROW_NUMBER()      | Ranking    | Unique sequential integer per row; no ties                                                     |
| RANK()            | Ranking    | Rank with gaps after ties                                                                      |
| DENSE_RANK()      | Ranking    | Rank without gaps after ties                                                                   |
| NTILE(n)          | Ranking    | Divides rows into n equal buckets                                                              |
| PERCENT_RANK()    | Ranking    | Relative rank as a percentage (0 to 1)                                                         |
| CUME_DIST()       | Ranking    | Cumulative distribution value (0 to 1)                                                         |
| SUM() OVER()      | Aggregate  | Running or grouped sum                                                                         |
| AVG() OVER()      | Aggregate  | Running or grouped average                                                                     |
| COUNT() OVER()    | Aggregate  | Running or grouped count                                                                       |
| MIN() OVER()      | Aggregate  | Minimum value in the frame                                                                     |
| MAX() OVER()      | Aggregate  | Maximum value in the frame                                                                     |
| LAG(col, n, def)  | Navigation | Value from n rows before current                                                               |
| LEAD(col, n, def) | Navigation | Value from n rows ahead of current                                                             |
| FIRST_VALUE(col)  | Navigation | First value in the window frame                                                                |
| LAST_VALUE(col)   | Navigation | Last value in the window frame                                                                 |
| NTH_VALUE(col, n) | Navigation | Value from the Nth row in the frame — NOT supported in SQL Server; use DENSE_RANK() workaround |

### Common Patterns Cheat Sheet

```
-- Running total
SUM(col) OVER (ORDER BY date_col ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

-- All-time / partition total
SUM(col) OVER (PARTITION BY group_col)

-- Rank within group
DENSE_RANK() OVER (PARTITION BY group_col ORDER BY measure DESC)
```

```
-- Previous row value
LAG(col) OVER (PARTITION BY group_col ORDER BY date_col)

-- Moving average (3 rows)
AVG(col) OVER (ORDER BY date_col ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

-- Top N per group pattern
WITH cte AS (SELECT *, RANK() OVER (PARTITION BY grp ORDER BY val DESC) rk FROM t)
SELECT * FROM cte WHERE rk <= N;
```

 **Note:** SQL Server Support: All functions in this guide work in SQL Server 2012+ EXCEPT NTH\_VALUE() (use DENSE\_RANK() workaround). DATEDIFF() in SQL Server requires a datepart as the first argument: DATEDIFF(DAY, start, end).